The Russian Talent Agency Problem: An Algorithmic Approach to Minimum-Cost Skill-Cover

Algorithm Design and Analysis
4th Year, Computer Science

Gabriel Andrés Pla Lasa
Carlos Daniel Largacha Leal

January 13, 2026

Abstract

This project addresses the Minimum-Cost Skill-Cover (MCSC) problem, a combinatorial optimization challenge arising in digital freelance platforms where the objective is to select an optimal team of freelancers satisfying project skill requirements at minimum total cost. We present a complete algorithmic analysis encompassing formalization, complexity classification, algorithm design, implementation, and empirical evaluation.

The problem is formally defined as an integer linear programming model with binary coverage constraints, capturing the essential features of real-world freelancer selection while maintaining analytical tractability. Through a polynomial-time reduction from the classical Set Cover problem, we prove MCSC is NP-complete, justifying the development of heuristic and approximate solution approaches.

We design and implement five algorithmic strategies: (1) an exact brute-force algorithm for small instances, (2) a greedy approximation algorithm based on cost-effectiveness ratios, (3) a local search improvement heuristic, (4) a genetic algorithm employing evolutionary operators, and (5) a novel hybrid approach combining kernelization preprocessing with adaptive algorithm selection.

Experimental evaluation on synthetic instances ranging from 10 to 1000 freelancers demonstrates that the hybrid algorithm achieves optimal or near-optimal solutions with exceptional efficiency, combining exact accuracy with metaheuristic scalability. Comparative analysis reveals that while greedy algorithms offer minimal execution time, they exhibit significant suboptimality in complex scenarios, whereas the hybrid approach adaptively selects the best strategy for the problem scale.

The primary contribution of this work is the demonstration that domain-specific preprocessing through kernelization can transform NP-hard optimization problems into tractable instances, enabling exact solutions on substantially reduced search spaces. This research provides both theoretical insights into the complexity of skill-cover problems and practical algorithmic solutions for real-world platform implementation.

Keywords: Minimum-Cost Skill-Cover, NP-completeness, combinatorial optimization, kernelization, hybrid algorithms, freelance platform optimization, set covering, genetic algorithms, approximation algorithms.

Contents

1 Introduction **4**
1.1 Motivation . . . 4
1.2 Objectives . . . 4
1.3 Document Organization . . . 5

2 Problem Formalization **5**
2.1 Definitions and Notation . . . 5
2.2 Mathematical Model . . . 6
2.3 Illustrative Example . . . 6

3 Computational Complexity Analysis **7**
3.1 Membership in NP . . . 7
3.2 Reduction from Set Cover . . . 8
3.3 Proof of NP-Completeness . . . 9
3.4 Complexity Implications . . . 9

4 Algorithm Design **10**
4.1 Exact Algorithm: Brute-Force with Backtracking . . . 10
4.1.1 Algorithm 1: Exhaustive Enumeration . . . 10
4.1.2 Coverage Verification Function . . . 10
4.1.3 Complexity Analysis . . . 11
4.1.4 Backtracking Optimization . . . 11
4.2 Approximation Algorithms . . . 11
4.2.1 Greedy Approximation Algorithm . . . 11
4.2.2 Approximation Guarantee . . . 12
4.3 Heuristic and Metaheuristic Approaches . . . 13
4.3.1 Local Search Algorithm . . . 13
4.3.2 Genetic Algorithm . . . 14
4.4 Kernelization Techniques . . . 14
4.5 Hybrid Algorithm . . . 15
4.6 Theoretical Analysis Summary . . . 15

5 Implementation and Experimental Analysis **16**
5.1 Implementation Details . . . 16
5.2 Experimental Methodology . . . 17
5.2.1 Instance Generation . . . 17
5.2.2 Performance Metrics . . . 17
5.3 Experimental Results . . . 17
5.3.1 Solution Quality Comparison . . . 17
5.3.2 Execution Time Analysis . . . 18
5.3.3 Kernelization Effectiveness in Hybrid Approach . . . 18
5.4 Discussion of Results . . . 19
5.5 Conclusion of Experimental Analysis . . . 19

6 Discussion **20**

7 Conclusions **20**
7.1 Main Findings 21
7.2 Contributions 21
7.3 Future Work 22
7.4 Final Remarks 22

1 Introduction

The rapid growth of the digital economy has fundamentally transformed labor markets, giving rise to expansive freelance platforms that connect specialized professionals with project-based opportunities. Within this context, platforms like TalentBridge Connect face a critical operational challenge: efficiently assembling optimal teams of freelancers to satisfy complex project requirements while minimizing costs. This team formation problem, though conceptually straightforward, reveals deep computational complexity when formalized mathematically, presenting significant algorithmic challenges that bridge theoretical computer science and practical software engineering.

1.1 Motivation

The proliferation of remote work and distributed teams has amplified the importance of algorithmic solutions for talent matching and team assembly. Traditional manual approaches to freelancer selection become infeasible as platform scales reach thousands of professionals and hundreds of simultaneous projects. The Minimum-Cost Skill-Cover (MCSC) problem encapsulates this real-world challenge, demanding solutions that balance computational efficiency with solution quality. Beyond immediate practical applications, this problem represents an interesting theoretical case study in combinatorial optimization, combining elements of set covering, weighted selection, and constraint satisfaction with domain-specific nuances like skill proficiency levels.

From an academic perspective, MCSC provides fertile ground for applying and extending techniques from algorithm design and computational complexity theory. Its structural similarity to classical NP-hard problems like Weighted Set Cover, combined with additional constraints involving skill levels, creates a rich problem space for algorithmic innovation. This project offers an opportunity to demonstrate how theoretical concepts in algorithm design translate to practical implementations addressing genuine business challenges.

1.2 Objectives

This project aims to conduct a comprehensive analysis of the Minimum-Cost Skill-Cover problem through the complete lifecycle of algorithmic problem-solving. The specific objectives are:

- **Formal Problem Definition:** To develop a precise mathematical model of the MCSC problem, transforming the informal business description into a well-defined computational problem with clear inputs, constraints, and optimization criteria.
- **Complexity Analysis:** To establish the computational complexity of MCSC through formal proof, demonstrating its NP-completeness via reduction from known hard problems, thereby justifying the need for heuristic and approximate solutions.
- **Algorithm Design:** To develop and implement multiple algorithmic strategies including exact methods for small instances, approximation algorithms with performance guarantees, and metaheuristic approaches for larger problem scales.
- **Empirical Evaluation:** To conduct systematic experimental analysis comparing algorithm performance across instance sizes, measuring both solution quality and

computational efficiency, and identifying the most effective approaches for different problem scales.

- **Practical Synthesis:** To provide actionable recommendations for platform implementation based on experimental findings, balancing theoretical insights with practical considerations for real-world deployment.

1.3 Document Organization

This report is structured to guide the reader through the complete analytical process applied to the MCSC problem. Section 2 establishes the formal mathematical model, defining the problem with precision and presenting an illustrative example. Section 3 provides the computational complexity analysis, proving NP-completeness through reduction from Set Cover. Section 4 details the algorithmic approaches designed and implemented, including exact, approximate, and metaheuristic strategies. Section 5 presents the experimental methodology and results, comparing algorithm performance across varying problem scales. Section 6 interprets the findings, discusses limitations, and suggests future research directions. Finally, Section 7 synthesizes the key insights and contributions of this investigation.

The appendices provide supplementary material including implementation details, source code references, and additional experimental data. This structured approach ensures a comprehensive treatment of the MCSC problem from theoretical foundations to practical implementation and evaluation.

2 Problem Formalization

The Minimum-Cost Skill-Cover (MCSC) problem involves the optimal selection of freelancers to satisfy project requirements at minimum cost. This section presents the complete mathematical formalization, transforming the business description into a precise computational model with well-defined components, constraints, and objectives.

2.1 Definitions and Notation

Let a system consist of a set of available freelancers and a specific project with skill requirements. We define the following sets and functions:

The set of freelancers is denoted by $F = \{f_1, f_2, \ldots, f_m\}$, where m represents the total number of freelancers on the platform. Each freelancer $f_i \in F$ has an associated cost $c_i \in \mathbb{R}^+$, corresponding to their hourly rate.

The universe of possible skills in the system is represented by $H = \{h_1, h_2, \ldots, h_p\}$, where p indicates the total number of skills that may be considered. For each freelancer f_i, we define a skill level function $\text{level}_i : H \to \mathbb{N}_0$ that assigns to each skill $h_j \in H$ a numerical value representing proficiency. The value $\text{level}_i(h_j) = 0$ indicates the freelancer does not possess skill h_j, while positive values represent varying competency levels.

The specific project defines a subset of required skills $R \subseteq H$. For each required skill $h_j \in R$, a minimum required level $\text{req}(h_j) \in \mathbb{N}^+$ is established, representing the minimum acceptable competency for that skill within the project context.

2.2 Mathematical Model

The solution to the problem consists of selecting a subset of freelancers $F' \subseteq F$ that satisfies all project requirements at minimum total cost. To formalize this notion, we introduce a binary decision variable x_i for each freelancer $f_i \in F$, defined as:

$$x_i = \begin{cases} 1 & \text{if freelancer } f_i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

The fundamental skill coverage condition establishes that for each required skill $h_j \in R$, there must exist at least one selected freelancer possessing that skill at a level equal to or exceeding the minimum requirement. Formally, this constraint is expressed as:

$$\forall h_j \in R: \quad \max_{i \in \{1,\ldots,m\}} \left\{ x_i \cdot \mathbb{I}_{[\text{level}_i(h_j) \geq \text{req}(h_j)]} \right\} = 1$$

where $\mathbb{I}_{[condition]}$ represents the indicator function taking value 1 if the condition is true and 0 otherwise.

An equivalent formulation, more convenient for algorithmic analysis, can be obtained by defining a binary coverage matrix $A = [a_{ij}]_{m \times |R|}$ where:

$$a_{ij} = \begin{cases} 1 & \text{if level}_i(h_j) \geq \text{req}(h_j) \\ 0 & \text{otherwise} \end{cases}$$

With this notation, the coverage constraints are expressed as:

$$\forall j \in \{1, \ldots, |R|\}: \quad \sum_{i=1}^{m} a_{ij} x_i \geq 1$$

The objective of the problem is to minimize the total cost of the selected team, leading to the following objective function:

$$\text{Minimize} \quad Z = \sum_{i=1}^{m} c_i x_i$$

The complete formulation of the problem as an integer linear program (ILP) is:

$$\begin{aligned} \underset{x_1,\ldots,x_m}{\text{minimize}} \quad & \sum_{i=1}^{m} c_i x_i \\ \text{subject to} \quad & \sum_{i=1}^{m} a_{ij} x_i \geq 1, \quad \forall j \in \{1, \ldots, |R|\} \\ & x_i \in \{0, 1\}, \quad \forall i \in \{1, \ldots, m\} \end{aligned}$$

2.3 Illustrative Example

Consider a scenario with three available freelancers $F = \{f_1, f_2, f_3\}$ and a project requiring three skills $R = \{\text{Frontend}, \text{Database}, \text{UX/UI}\}$. The freelancers' characteristics and project requirements are detailed below.

The freelancers exhibit the following costs and skill profiles:

- f_1: Cost 50, skills $\{\text{Frontend:4}, \text{Database:2}\}$

- f_2: Cost 40, skills {Database:4, UX/UI:3}
- f_3: Cost 60, skills {Frontend:5, UX/UI:5}

The project establishes the following minimum requirements: Frontend ≥ 4, Database ≥ 3, and UX/UI ≥ 4.

The resulting coverage matrix A is:

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Possible solutions and their costs are:

- $\{f_1, f_2, f_3\}$: Cost 150 - Covers all skills
- $\{f_2, f_3\}$: Cost 100 - Covers all skills (f_2 covers DB, f_3 covers Frontend and UX/UI)
- $\{f_1, f_3\}$: Cost 110 - Does not cover DB (f_1 has level 2 ¡ 3)
- $\{f_1, f_2\}$: Cost 90 - Does not cover UX/UI (f_2 has level 3 ¡ 4)

The optimal solution is $\{f_2, f_3\}$ with total cost 100, satisfying all project requirements at minimum possible cost.

This formalization completely captures the essence of the problem described in the TalentBridge Connect context, providing a solid foundation for the complexity analysis and algorithm design presented in subsequent sections. The model enables representation of arbitrary problem instances and establishes the framework for evaluating different algorithmic approaches.

3 Computational Complexity Analysis

This section demonstrates that the Minimum-Cost Skill-Cover Problem (MCSC) belongs to the class of NP-complete problems. We formally present the decision version of the problem, prove its membership in NP, and provide a polynomial-time reduction from the classical Set Cover problem, thereby establishing its NP-completeness.

3.1 Membership in NP

Definition 3.1 (Minimum-Cost Skill-Cover - Decision Version) *Let an instance of MCSC be defined by:*

- *A set of freelancers* $F = \{f_1, f_2, \ldots, f_m\}$
- *For each* $f_i \in F$*: a cost* $c_i \in \mathbb{Z}^+$ *and a skill level function* $level_i : H \to \mathbb{N}_0$
- *A set of required skills* $R \subseteq H$
- *For each* $h_j \in R$*: a minimum required level* $req(h_j) \in \mathbb{N}^+$
- *An integer* $K \geq 0$

The decision question is: Does there exist a subset $F' \subseteq F$ *such that:*

1. *For each skill $h_j \in R$, there exists at least one freelancer $f_i \in F'$ with $level_i(h_j) \geq req(h_j)$*

2. *The sum of costs $\sum_{f_i \in F'} c_i \leq K$?*

Proposition 3.1 *The decision version of MCSC belongs to the class NP.*

Proof 1 *To demonstrate that MCSC $\in$ NP, we must exhibit a polynomial-time verifier. Given a certificate consisting of a subset $F' \subseteq F$, the verifier performs the following checks:*

1. *For each skill $h_j \in R$, verify that there exists some $f_i \in F'$ with $level_i(h_j) \geq req(h_j)$. This verification requires time $O(|F'| \cdot |R|)$.*

2. *Calculate the total cost $\sum_{f_i \in F'} c_i$ and compare with K, which requires time $O(|F'|)$.*

The total verification time is $O(|F'| \cdot |R|)$, bounded by $O(m \cdot n)$ where $m = |F|$ and $n = |R|$. Therefore, a polynomial-time verifier exists, and MCSC $\in$ NP.

3.2 Reduction from Set Cover

Definition 3.2 (Set Cover (SC)) *Given a universe $U = \{u_1, u_2, \ldots, u_n\}$, a collection $S = \{S_1, S_2, \ldots, S_m\}$ of subsets of U, and an integer t, the Set Cover problem consists of determining whether there exists a subcollection $C \subseteq S$ with $|C| \leq t$ such that $\bigcup_{S_i \in C} S_i = U$.*

Set Cover is a classical NP-complete problem, originally established by Karp in his seminal 1972 work [1] that identified 21 fundamental NP-complete problems. This classification provides the theoretical foundation for our reduction and subsequent algorithm design decisions.

Theorem 3.1 *There exists a polynomial-time reduction from Set Cover to Minimum-Cost Skill-Cover.*

Proof 2 *Let $I_{SC} = (U, S, t)$ be an arbitrary instance of Set Cover, where $U = \{u_1, u_2, \ldots, u_n\}$, $S = \{S_1, S_2, \ldots, S_m\}$ with $S_i \subseteq U$, and $t \in \mathbb{Z}^+$.*

We construct an instance $I_{MCSC} = (F, H, R, level, req, c, K)$ of MCSC as follows:

1. *Define the set of skills $H = U$.*

2. *Establish that all skills are required: $R = H = U$.*

3. *For each set $S_i \in S$, create a freelancer $f_i \in F$ with:*

   - *Cost $c_i = 1$*
   - *Skill level function $level_i : H \to \{0, 1\}$ defined as:*

$$level_i(h_j) = \begin{cases} 1 & \text{if } u_j \in S_i \\ 0 & \text{if } u_j \notin S_i \end{cases}$$

   where $h_j \in H$ corresponds to element $u_j \in U$.

4. *For each skill $h_j \in R$, set $req(h_j) = 1$.*

5. *Define $K = t$.*

The construction of I_{MCSC} requires time $O(m \cdot n)$, which is polynomial in the size of I_{SC}.

3.3 Proof of NP-Completeness

Lemma 3.1 *The instance I_{SC} of Set Cover has a solution with size $\leq t$ if and only if the instance I_{MCSC} of MCSC has a solution with cost $\leq K$.*

Proof 3 *We prove both directions of the equivalence.*

*(⇒) **Forward direction:** Suppose $C \subseteq S$ is a set cover for I_{SC} with $|C| \leq t$. Construct $F' = \{f_i \in F : S_i \in C\}$. Each $f_i \in F'$ has cost $c_i = 1$, so the total cost is $|F'| = |C| \leq t = K$.*

To verify that F' is a valid team, let $h_j \in R$ be any skill. Since C is a set cover, there exists $S_i \in C$ such that $u_j \in S_i$. By construction, the corresponding freelancer f_i has $level_i(h_j) = 1$. Given that $req(h_j) = 1$, we have $level_i(h_j) \geq req(h_j)$. Therefore, h_j is covered by $f_i \in F'$, and F' is a solution for I_{MCSC}.

*(⇐) **Reverse direction:** Suppose $F' \subseteq F$ is a team for I_{MCSC} with total cost $\leq K$. Construct $C = \{S_i \in S : f_i \in F'\}$. Since each f_i has cost 1, $|C| = |F'| \leq K = t$.*

To verify that C is a set cover, let $u_j \in U$ be any element. The corresponding skill h_j is in R. Since F' is a valid team, there exists $f_i \in F'$ such that $level_i(h_j) \geq req(h_j) = 1$. By construction, $level_i(h_j) = 1$ implies $u_j \in S_i$, and therefore $S_i \in C$ covers u_j. Thus, C is a set cover for I_{SC}.

Theorem 3.2 *Minimum-Cost Skill-Cover is NP-complete.*

Proof 4 *The proof follows directly from the established results:*

1. *By Proposition 1, MCSC ∈ NP.*
2. *By Theorem 1 and Lemma 1, MCSC is NP-hard, since Set Cover (NP-complete) reduces polynomially to MCSC.*
3. *Therefore, MCSC is NP-complete.*

3.4 Complexity Implications

Corollary 3.1 *The optimization version of MCSC (finding the minimum-cost team) is NP-hard.*

Proof 5 *If a polynomial-time algorithm existed for the optimization version, the decision version could be solved in polynomial time by comparing the minimum cost obtained with the threshold K.*

Proposition 3.2 (Complexity of Exhaustive Algorithm) *The brute-force algorithm for MCSC, which evaluates all possible subsets of freelancers, has time complexity $O(2^m \cdot m \cdot n)$, where $m = |F|$ and $n = |R|$.*

Proof 6 *The search space consists of all 2^m possible subsets of F. For each candidate subset $F' \subseteq F$, the algorithm must:*

1. *Verify that for each $h_j \in R$ there exists $f_i \in F'$ with $level_i(h_j) \geq req(h_j)$, requiring time $O(|F'| \cdot |R|) \leq O(m \cdot n)$.*
2. *Calculate the total cost $\sum_{f_i \in F'} c_i$, requiring time $O(|F'|) \leq O(m)$.*

The total complexity is therefore $O(2^m \cdot m \cdot n)$, an exponential function in m.

The NP-completeness of MCSC has important practical consequences. First, it justifies the use of non-exact algorithms for substantial instances, as no polynomial-time exact algorithm is expected to exist (unless P = NP). Second, it provides theoretical foundation for the algorithmic approaches presented in the following section, which include approximation algorithms and metaheuristics designed to deliver high-quality solutions within practical time constraints.

4 Algorithm Design

Given the NP-hardness of the Minimum-Cost Skill-Cover Problem established in the previous section, this section presents a comprehensive algorithmic framework addressing both exact and approximate solution strategies. We design two categories of algorithms: an exact brute-force algorithm for small instances serving as a baseline, and several approximate algorithms for practical large-scale instances.

4.1 Exact Algorithm: Brute-Force with Backtracking

The exact algorithm employs a systematic exploration of all possible freelancer subsets. The fundamental approach consists of evaluating each subset of F, verifying coverage of all required skills, and selecting the minimum-cost valid team.

4.1.1 Algorithm 1: Exhaustive Enumeration

Algorithm 1 Exact Brute-Force Algorithm for MCSC

Require: Set of freelancers $F = \{f_1, \ldots, f_m\}$ with costs c_i and skill levels $\text{level}_i(h_j)$; Project requirements $R \subseteq H$ with minimum levels $\text{req}(h_j)$
Ensure: Optimal team $F^* \subseteq F$ and its cost C^*

```
best_cost ← ∞
best_team ← ∅
for b = 0 to 2^m − 1 do
    current_team ← {f_i ∈ F | bit i of b = 1}
    current_cost ← Σ_{f_i ∈ current_team} c_i
    if current_cost ≥ best_cost then
        continue                                   ▷ Cost-based pruning
    end if
    if covers_all_requirements(current_team, R) then
        best_cost ← current_cost
        best_team ← current_team
    end if
end for
return best_team, best_cost
```

4.1.2 Coverage Verification Function

The auxiliary function covers_all_requirements(F', R) determines whether a candidate team F' satisfies all project requirements. In mathematical terms:

$$\text{covers\_all\_requirements}(F', R) = \begin{cases} \text{true} & \text{if } \forall h_j \in R, \exists f_i \in F' : \text{level}_i(h_j) \geq \text{req}(h_j) \\ \text{false} & \text{otherwise} \end{cases}$$

Note that the indicator function $\mathbb{I}_{[\text{condition}]}$ takes the value 1 if the condition is true and 0 otherwise.

4.1.3 Complexity Analysis

The exhaustive enumeration algorithm has time complexity $O(2^m \cdot m \cdot n)$, where $m = |F|$ is the number of freelancers and $n = |R|$ is the number of required skills. The space complexity is $O(m + n)$ for storing the current team and tracking coverage.

4.1.4 Backtracking Optimization

A more efficient implementation uses backtracking with pruning strategies:

Algorithm 2 Backtracking with Pruning

```
function BACKTRACK(F, idx, current_team, current_cost, covered_skills)
    if covered_skills = R then
        Update best solution if current_cost is better
        return
    end if
    if idx = |F| then
        return
    end if
                                                ▷ Option 1: Skip current freelancer
    BACKTRACK(F, idx + 1, current_team, current_cost, covered_skills)
                                        ▷ Option 2: Include freelancer if beneficial
    new_skills ← {h ∈ R \ covered_skills | F[idx].level(h) ≥ req(h)}
    if new_skills ≠ ∅ then
        BACKTRACK(F, idx + 1, current_team ∪ {F[idx]}, current_cost + F[idx].cost,
    covered_skills ∪ new_skills)
    end if
end function
```

4.2 Approximation Algorithms

4.2.1 Greedy Approximation Algorithm

The greedy algorithm iteratively selects freelancers based on a cost-effectiveness ratio, prioritizing those covering many uncovered skills at low cost.

Algorithm 3 Greedy Approximation Algorithm

Require: F, R as defined previously
Ensure: Approximate team $\tilde{F} \subseteq F$

team $\leftarrow \emptyset$
covered_skills $\leftarrow \emptyset$
available_freelancers $\leftarrow F$
while covered_skills $\neq R$ **do**
    best_ratio $\leftarrow -\infty$
    best_freelancer $\leftarrow$ None
    for each $f \in$ available_freelancers **do**
        new_skills $\leftarrow \{h \in R \setminus \text{covered\_skills} \mid f.\text{level}(h) \geq \text{req}(h)\}$
        if $|\text{new\_skills}| > 0$ **then**
            ratio $\leftarrow |\text{new\_skills}|/f.\text{cost}$
            if ratio $>$ best_ratio **then**
                best_ratio $\leftarrow$ ratio
                best_freelancer $\leftarrow f$
            end if
        end if
    end for
    if best_freelancer = None **then**
        return "No feasible solution exists"
    end if
    team $\leftarrow$ team $\cup \{$best_freelancer$\}$
    covered_skills $\leftarrow$ covered_skills $\cup \{h \in R \mid \text{best\_freelancer.level}(h) \geq \text{req}(h)\}$
    available_freelancers $\leftarrow$ available_freelancers $\setminus \{$best_freelancer$\}$
end while
return team

4.2.2 Approximation Guarantee

The greedy algorithm described in Section 4.2.1 is a well-established approximation strategy for covering problems, with formal performance guarantees derived from its application to the classical Weighted Set Cover problem [2] [5]. In that canonical formulation, where each set $S_i \subseteq U$ has an associated cost c_i and the objective is to cover all elements in the universe U at minimum total cost, the greedy algorithm—which iteratively selects the set with the best cost-effectiveness ratio—achieves an approximation factor of $H(\max_i |S_i|)$, where $H(k) = \sum_{i=1}^{k} \frac{1}{i}$ is the k-th harmonic number. Since $H(k) \leq 1 + \ln k$, this yields an approximation ratio of $O(\log n)$ for an instance with $n = |U|$ elements to cover.

The Minimum-Cost Skill-Cover problem can be viewed as a generalization of Weighted Set Cover, where each element corresponds to a required skill, and each set corresponds to the subset of skills a freelancer covers at or above the required proficiency level. The greedy algorithm applied to MCSC follows the same principle of cost-effectiveness, selecting in each iteration the freelancer that covers the largest number of yet-uncovered skills per unit cost. Although the presence of skill levels introduces additional structure not present in the classic Set Cover, the underlying combinatorial nature of the coverage constraints remains unchanged. Consequently, the same approximation guarantees apply, provided that the coverage matrix a_{ij} correctly reflects whether a freelancer satisfies the minimum

level requirement $\text{req}(h_j)$ for each skill.

It is important to note that this $O(\log n)$ guarantee is a worst-case bound. In practice, the greedy algorithm often performs significantly better, particularly on instances where skill distributions and costs are reasonably balanced. However, as demonstrated experimentally in Section 5, there exist adversarial instances—such as the $m = 20$ case—where the greedy heuristic deviates from optimality due to its myopic decision-making. This behavior is consistent with the theoretical limitations of greedy algorithms for covering problems, where locally optimal choices can lead to globally suboptimal solutions.

4.3 Heuristic and Metaheuristic Approaches

4.3.1 Local Search Algorithm

Local search begins with an initial solution (typically from the greedy algorithm) and iteratively improves it through neighborhood exploration.

Algorithm 4 Local Search Improvement

Require: Initial solution initial_solution, F, R
Ensure: Improved solution

current_solution $\leftarrow$ initial_solution
improved $\leftarrow$ true
while improved **do**
    improved $\leftarrow$ false
    for each pair $(f_1, f_2) \in$ current_solution **do**
        for each $f_3 \in F \setminus$ current_solution **do**
            if f_3 covers all skills covered by $\{f_1, f_2\}$ **then**
                if $f_3.\text{cost} < f_1.\text{cost} + f_2.\text{cost}$ **then**
                    current_solution $\leftarrow$ (current_solution $\setminus \{f_1, f_2\}) \cup \{f_3\}$
                    improved $\leftarrow$ true
                    break
                end if
            end if
        end for
        if improved **then**
            break
        end if
    end for
    for each $f \in$ current_solution **do**
        if current_solution $\setminus \{f\}$ covers R **then**
            current_solution $\leftarrow$ current_solution $\setminus \{f\}$
            improved $\leftarrow$ true
            break
        end if
    end for
end while
return current_solution

4.3.2 Genetic Algorithm

The genetic algorithm employs evolutionary principles to explore the solution space through selection, crossover, and mutation operations, following established metaheuristic frameworks [3, 6]. This population-based approach enables escape from local optima that trap deterministic methods, providing robust exploration capabilities for complex combinatorial landscapes.

Algorithm 5 Genetic Algorithm for MCSC

Require: Population size P, maximum generations G, mutation rate μ
Ensure: Best solution found
Initialize population with P random solutions (or greedy solutions)
Evaluate fitness: $\text{fitness}(sol) = \frac{1}{\text{cost}(sol)+\lambda \cdot \text{penalty}(sol)}$
for $generation = 1$ **to** G **do**
    Select parents using tournament selection
    Apply crossover: union of parent teams followed by reduction
    Apply mutation: with probability μ, add or remove random freelancer
    Repair infeasible solutions using greedy completion
    Replace population maintaining elitism
end for
return Best solution in final population

4.4 Kernelization Techniques

Kernelization reduces problem instances by applying sound reduction rules that preserve optimal solutions.

Algorithm 6 Preprocessing via Kernelization

function PREPROCESS(F, R)
    mandatory $\leftarrow \emptyset$
    for each $h \in R$ **do**
        candidates $\leftarrow \{f \in F \mid f.\text{level}(h) \geq \text{req}(h)\}$
        if $|\text{candidates}| = 1$ **then**
            mandatory $\leftarrow$ mandatory $\cup$ candidates
        end if
    end for
    $R' \leftarrow R \setminus \{h \in R \mid h$ covered by mandatory$\}$
    $F' \leftarrow F \setminus$ mandatory
    for each $f_1 \in F'$ **do**
        for each $f_2 \in F' \setminus \{f_1\}$ **do**
            if f_1 dominates f_2 **then** ▷ f_1.cost $\leq f_2$.cost and f_1 covers superset of f_2's skills
                $F' \leftarrow F' \setminus \{f_2\}$
            end if
        end for
    end for
    return F', R', mandatory
end function

4.5 Hybrid Algorithm

Based on empirical observations, we propose a hybrid strategy that adapts to instance characteristics:

Algorithm 7 Adaptive Hybrid Algorithm

Require: F, R
Ensure: High-quality team
1: $F', R', \text{mandatory} \leftarrow \textsc{Preprocess}(F, R)$
2: solution $\leftarrow$ mandatory
3: **if** $|F'| \leq 20$ **then**
4: Apply exact backtracking on F'
5: **else if** $20 < |F'| \leq 100$ **then**
6: greedy_sol $\leftarrow \textsc{Greedy}(F', R')$
7: solution $\leftarrow$ solution $\cup \textsc{LocalSearch}(\text{greedy\_sol}, F', R')$
8: **else**
9: solution $\leftarrow$ solution $\cup \textsc{GeneticAlgorithm}(F', R')$
10: **end if**
11: **return** solution

The hybrid approach integrates kernelization techniques with adaptive algorithm selection based on instance size thresholds, following principles of algorithm design that emphasize problem decomposition and appropriate technique selection [4]. This approach exemplifies the algorithmic engineering philosophy of combining multiple techniques to address different aspects of complex problems.

4.6 Theoretical Analysis Summary

Table 1 summarizes the theoretical properties of the proposed algorithms.

Table 1: Theoretical Comparison of Proposed Algorithms

Algorithm	**Time Complexity**	**Solution Quality**	**Application Scope**
Exact Brute-Force	$O(2^m \cdot m \cdot n)$	Optimal	Small instances ($m \leq 20$)
Greedy Approximation	$O(m^2 \cdot n)$	$O(\log n)$-approximation	Fast decent solutions
Local Search	$O(k \cdot m^2 \cdot n)$	Improves initial solution	Post-processing refinement
Genetic Algorithm	$O(P \cdot G \cdot m \cdot n)$	High-quality exploration	Large-scale instances
Hybrid Adaptive	Variable	Balanced trade-off	General-purpose deployment

The algorithm design presented in this section provides a comprehensive toolkit for addressing MCSC instances of varying sizes and characteristics. The exact algorithm serves

as a baseline for validation, while the approximate algorithms offer practical solutions for real-world applications where optimality must be traded for computational efficiency.

5 Implementation and Experimental Analysis

This section presents the empirical evaluation of the proposed algorithms for the Minimum-Cost Skill-Cover (MCSC) problem, detailing implementation specifics, experimental methodology, and comprehensive performance analysis.

5.1 Implementation Details

The algorithmic framework was implemented in Python 3.10+ utilizing standard libraries, ensuring high portability without external dependencies. The codebase follows a modular architecture comprising several key components. The `main.py` module serves as the primary entry point for demonstrations, while `comp.py` handles systematic benchmarking and table generation. Algorithm implementations reside within the `algorithms/` directory, containing specialized modules for brute-force search, greedy approximation, local search, genetic algorithms, and the hybrid approach. Data abstractions for freelancers and projects are defined in the `models/` package, and the `utility/` module provides pseudo-random instance generation capabilities.

Table 2 details the specific parameter configurations employed for each algorithm during experimentation. The brute-force implementation incorporates cost and feasibility pruning strategies, executing exclusively for instances with $m \leq 20$ freelancers. The greedy algorithm employs a cost-effectiveness heuristic based on the ratio of newly covered skills to freelancer cost. Local search operates through a hill-climbing strategy with first-improvement acceptance, utilizing redundancy elimination and one-to-one swap operators. The genetic algorithm maintains a population of 50 individuals across 100 generations, applying uniform crossover with 0.2 mutation probability and tournament selection ($k = 3$). The hybrid approach integrates kernelization techniques with adaptive algorithm selection based on instance size thresholds.

Table 2: Algorithm Parameters Configuration

Algorithm	**Parameters and Values**
Brute-Force	Pruning: Cost and feasibility bounds. Execution limited to $m \leq 20$.
Greedy	Heuristic: Maximum ratio of new skills to cost.
Local Search	Strategy: Hill Climbing (First Improvement). Operators: Redundancy elimination, 1-1 swap.
Genetic Algorithm	Population: 50. Generations: 100. Mutation probability: 0.2. Selection: Tournament ($k = 3$). Crossover: Uniform.
Hybrid Algorithm	Kernelization: Dominance and essentiality rules. Thresholds: $m \leq 20$ (brute force), $20 < m \leq 100$ (greedy + local search), $m > 100$ (genetic).

5.2 Experimental Methodology

5.2.1 Instance Generation

A pseudo-random instance generator was developed to create reproducible test cases simulating realistic market conditions. The generator parameters, detailed in Table 3, were calibrated to produce diverse problem instances. Freelancer costs follow a uniform distribution between 20 and 100 abstract currency units, representing varied hourly rates. Each freelancer possesses any given skill with probability $p = 0.4$, with skill proficiency levels uniformly distributed between 1 and 5. Project requirements specify minimum skill levels drawn uniformly from the range 1 to 3.

Table 3: Instance Generation Parameters

Parameter	**Range/Distribution**	**Description**
Freelancer Cost	Uniform $[20, 100]$	Hourly rates in abstract units
Skill Probability	$p = 0.4$	Probability freelancer possesses skill
Skill Levels	Uniform $[1, 5]$	Proficiency level of freelancer
Requirements	Uniform $[1, 3]$	Minimum required skill level

5.2.2 Performance Metrics

Algorithm performance was evaluated using three principal metrics. Solution cost measures the total expenditure of the selected team, with lower values indicating superior performance. Execution time captures wall-clock computational requirements in seconds. For the hybrid approach, the reduction rate quantifies the percentage of freelancers eliminated during kernelization preprocessing.

5.3 Experimental Results

Experiments were conducted across four instance scales: $m \in \{10, 20, 50, 150\}$ freelancers. Each configuration was evaluated using all applicable algorithms, with results averaged over multiple runs to ensure statistical reliability.

5.3.1 Solution Quality Comparison

Table 4 presents the comparative solution costs obtained by each algorithm. For the smallest instance ($m = 10$), all algorithms successfully identify the optimal solution at cost 108.0, indicating the tractability of trivial problem sizes. At $m = 20$, a notable performance divergence emerges: while brute-force establishes the optimal cost at 77.0, the greedy algorithm and local search produce suboptimal solutions at 84.0, representing a 9.1% optimality gap. Both genetic and hybrid algorithms match the brute-force optimum at this scale. For larger instances ($m = 50, 150$) where brute-force becomes computationally prohibitive, the genetic and hybrid algorithms demonstrate superior performance, achieving cost 41.0 at $m = 150$ compared to 43.0 from greedy and local search approaches.

Table 4: Solution Cost Comparison

m	Brute-Force	Greedy	Local Search	Genetic	Hybrid
10	52.0	52.0	52.0	52.0	52.0
20	149.0	149.0	149.0	149.0	149.0
50	96.0	116.0	96.0	96.0	96.0
150	56.0	56.0	56.0	56.0	56.0
300	67.0	67.0	67.0	67.0	67.0
500	67.0	81.0	67.0	73.0	67.0
750	65.0	84.0	74.0	65.0	65.0
1000	N/A	83.0	68.0	86.0	81.0

5.3.2 Execution Time Analysis

Computational efficiency results appear in Table 5. The greedy algorithm exhibits consistently minimal execution times across all instance sizes, typically below 0.0001 seconds. Local search maintains similar efficiency, requiring only marginally more computation. The genetic algorithm demonstrates higher constant overhead, approximately 0.02 seconds per instance, though this cost remains stable regardless of problem scale. Most notably, the hybrid approach achieves exceptional efficiency, solving the largest instance ($m = 150$) in merely 0.0006 seconds while maintaining solution quality. This represents a 38-fold speed advantage over the genetic algorithm at equivalent quality levels.

Table 5: Execution Time Comparison (seconds)

m	Brute-Force	Greedy	Local Search	Genetic	Hybrid
10	0.0001	0.0000	0.0000	0.0166	0.0001
20	0.0005	0.0000	0.0001	0.0211	0.0002
50	0.0028	0.0001	0.0002	0.0223	0.0007
150	0.0506	0.0003	0.0004	0.0300	0.0039
300	0.7608	0.0011	0.0015	0.0533	0.2966
500	4.6766	0.0033	0.0040	0.0673	2.4421
750	18.7862	0.0054	0.0067	0.1189	15.4450
1000	N/A	0.0213	0.0360	0.4829	0.8506

5.3.3 Kernelization Effectiveness in Hybrid Approach

The superior performance of the hybrid algorithm stems primarily from its preprocessing phase. Table 6 quantifies kernelization effectiveness across instance sizes. Remarkably, the reduction process eliminates 70-92% of freelancers through dominance and essentiality analysis. For $m = 150$, the candidate pool reduces from 150 to merely 12 freelancers, enabling exact brute-force resolution on the reduced instance in negligible time. This preprocessing not only ensures optimality guarantees but also explains the hybrid algorithm's exceptional speed compared to both pure exact and approximate methods.

Table 6: Kernelization Effectiveness in Hybrid Algorithm

m	Reduced m	Reduction %	Solver Used	Result
10	2	80%	Brute Force	Optimum
20	8	60%	Brute Force	Optimum
50	17	66%	Brute Force	Optimum
150	44	71%	Brute Force	Optimum
300	199	34%	Brute Force	Optimum
500	380	24%	Brute Force	Optimum
750	644	14%	Brute Force	Optimum
1000	938	6%	Genetic	Best found

5.4 Discussion of Results

The experimental outcomes reveal several significant patterns regarding algorithm behavior and effectiveness. The greedy algorithm's failure at $m = 20$ illustrates a fundamental limitation of locally optimal strategies when confronted with complex trade-offs between specialized and generalist freelancers. Analysis of this instance reveals structural properties where no single freelancer provides dominant cost-coverage ratios, creating solution landscapes with deceptive local optima.

Local search exhibits similar limitations, frequently stagnating at solutions identical to those produced by greedy initialization. This suggests insufficient neighborhood exploration or inadequate diversification mechanisms within the implemented hill-climbing framework. The genetic algorithm demonstrates superior exploration capabilities through its population-based approach, successfully escaping local optima that trap deterministic methods.

The hybrid approach's performance derives from its synergistic combination of preprocessing and algorithmic adaptation. Local Search, recently enhanced with the **2-1 Swap operator**, has shown remarkable power: it now finds optimal costs (67.0) for $m = 500$ and even outperforms the Genetic Algorithm at $m = 1000$ (68.0 vs 86.0). Kernelization rules effectively exploit problem structure to eliminate dominated candidates. While the reduction rate is exceptional for smaller instances (92% for $m = 150$), it naturally decreases as requirements become denser, reaching 6% for $m = 1000$. The significant quality gap observed (Greedy 83 vs LS 68 at $m = 1000$) reinforces the value of sophisticated neighborhood search.

5.5 Conclusion of Experimental Analysis

Empirical evaluation validates the hybrid algorithm as the most effective approach for the MCSC problem, successfully balancing solution quality with computational efficiency. The methodology demonstrates that problem-specific preprocessing through kernelization can dramatically reduce search space complexity, enabling exact solutions on instances as large as $m = 750$ when combined with backtracking. For mass-scale instances ($m = 1000$), the algorithm adaptively switches to metaheuristics, still achieving superior results compared to simple greedy approaches. These findings emphasize that as constraints grow, the combination of reduction rules and robust optimization becomes critical for maintaining solution quality.

6 Discussion

The experimental analysis yields three principal insights with significant implications for both theoretical understanding and practical application of the Minimum-Cost Skill-Cover problem.

The algorithmic performance patterns reveal fundamental distinctions between solution strategies. The failure of greedy and local search approaches at $m = 20$, contrasted with the success of genetic and hybrid methods, indicates the presence of deceptive local optima that trap simpler heuristics. This phenomenon exemplifies the classic limitation of hill-climbing strategies in combinatorial optimization [4]. More significantly, the hybrid algorithm's exceptional performance demonstrates that kernelization preprocessing can transform NP-hard instances into tractable subproblems, achieving 92% reduction for $m = 150$ while preserving optimality guarantees. The successful application of kernelization techniques adapted from parameterized complexity theory demonstrates that practical instances may exhibit structural properties amenable to significant reduction despite theoretical hardness [5]. This aligns with algorithmic design principles that advocate for exploiting problem-specific structure whenever possible [4].

Practical deployment considerations emerge from the efficiency-quality trade-offs observed. The hybrid approach provides optimal or near-optimal solutions across all ranges. For $m \leq 300$, it is extremely fast; however, for $m = 750$, it requires $\approx 34s$ to guarantee optimality through its exact phase. This demonstrates that while kernelization is powerful, the exponential nature of MCSC eventually reappears in very large, dense instances, justifying the hybrid switch to a genetic algorithm at $m = 1000$ to maintain a 2-second response time.

Several methodological limitations contextualize these findings. The synthetic instance generation may not fully capture real-world skill correlations and cost distributions. The exclusive focus on cost minimization neglects practical factors such as team coordination and freelancer reliability. Additionally, the Python implementation, while suitable for experimentation, introduces performance characteristics that may differ from production environments using compiled languages.

Future research directions include extending kernelization with additional reduction rules, developing more sophisticated local search neighborhoods, and exploring multi-objective formulations that consider secondary criteria like team size and skill redundancy. The algorithmic framework developed here also shows potential for generalization to related problems in team formation and resource-constrained selection.

In conclusion, this research demonstrates that despite theoretical NP-hardness, practical MCSC instances can be solved effectively through algorithms that exploit domain-specific structure. The hybrid approach's success underscores the value of combining theoretical insight with practical engineering in addressing complex combinatorial optimization problems.

7 Conclusions

This project has conducted a comprehensive investigation of the Minimum-Cost Skill-Cover (MCSC) problem, addressing it through the complete lifecycle of algorithmic problem-solving: from formal definition and complexity analysis to algorithm design, implementation, and empirical evaluation. The research demonstrates how theoretical computer science principles can be effectively applied to solve practical optimization challenges in

modern digital labor markets.

7.1 Main Findings

The formalization of MCSC established a precise mathematical model that captures the essential features of real-world freelancer selection while maintaining analytical tractability. The formulation as an integer linear program with binary coverage constraints provides both clarity for theoretical analysis and a foundation for algorithmic development.

The complexity analysis conclusively established MCSC as NP-complete through a polynomial-time reduction from the classical Set Cover problem. This theoretical result has significant practical implications: it justifies the development of heuristic and approximate algorithms for practical problem instances while explaining the computational intractability of exact approaches for large-scale scenarios.

The experimental evaluation revealed distinct performance characteristics across the implemented algorithms. The brute-force approach, while optimal for small instances ($m \leq 20$), becomes computationally prohibitive beyond this threshold. The greedy algorithm demonstrates exceptional speed but exhibits suboptimal performance in scenarios with complex trade-offs, achieving only 91% of optimal quality at $m = 20$. Local search shows limited improvement over greedy solutions, suggesting insufficient exploration capabilities. The genetic algorithm provides consistent quality across instance sizes but with higher computational overhead. Most significantly, the hybrid algorithm achieves optimal or near-optimal solutions with minimal execution time by effectively combining kernelization preprocessing with adaptive algorithm selection.

7.2 Contributions

This work makes several contributions to both theoretical understanding and practical application. The formal proof of NP-completeness for MCSC establishes its position within the computational complexity hierarchy and provides theoretical justification for heuristic approaches. The development of a kernelization-based hybrid algorithm demonstrates how domain-specific preprocessing can transform NP-hard problems into tractable instances, achieving up to 92% reduction in problem size while preserving optimality guarantees.

The systematic experimental framework, including a configurable instance generator and comprehensive performance metrics, creates a reproducible foundation for further research on skill-cover problems. The comparative analysis of algorithmic approaches provides empirical evidence of their relative strengths and limitations across varying problem scales, offering practical guidance for platform implementers.

From a methodological perspective, the project exemplifies the complete algorithmic problem-solving process: from informal problem description through mathematical formalization, complexity analysis, algorithm design, implementation, and empirical validation. This end-to-end approach demonstrates how theoretical computer science principles can be effectively translated into practical solutions for real-world optimization challenges.

Based on the experimental findings, we recommend the hybrid algorithm as the primary solution approach for operational deployment. Its combination of kernelization preprocessing with adaptive algorithm selection achieves the best known balance between solution quality and computational efficiency. The algorithm's ability to maintain optimal results up to $m = 750$ and its graceful transition to metaheuristics for larger scales makes it ideal for platforms requiring reliability at scale.

For practical implementation, we suggest the following configuration:

- Implement kernelization as a mandatory preprocessing phase to eliminate dominated freelancers
- Use exact methods for instances reduced to 20 or fewer candidates
- Apply greedy algorithms with local search refinement for medium-sized instances
- Employ genetic algorithms for complex instances where kernelization provides limited reduction
- Incorporate periodic validation using multiple algorithms to ensure solution quality

The kernelization phase not only accelerates computation but also provides valuable business intelligence by identifying freelancers who are consistently dominated in the market, potentially informing platform recommendations and skill development guidance.

7.3 Future Work

Several promising directions for future research emerge from this investigation. Extending the kernelization phase with additional reduction rules based on linear programming relaxations or subset cover relationships could further improve preprocessing effectiveness. Developing more sophisticated local search mechanisms with adaptive neighborhood structures and tabu search elements might enhance solution refinement capabilities.

Exploring multi-objective formulations that consider additional criteria such as team size minimization, skill coverage redundancy, freelancer reliability ratings, or temporal availability constraints would better capture real-world decision factors. Investigating online versions of the problem, where freelancer availability and project requirements evolve dynamically, represents another valuable extension.

The algorithmic framework developed for MCSC shows potential for generalization to related problems in team formation, resource-constrained selection, and service composition. Future work could explore these connections and develop unified approaches for this class of covering and selection problems.

Finally, validating the algorithms on real-world data from freelance platforms would provide important insights into their practical performance and identify opportunities for domain-specific optimizations not captured in synthetic instances.

7.4 Final Remarks

This research demonstrates that despite theoretical NP-hardness, practical instances of the Minimum-Cost Skill-Cover problem can be solved effectively through careful algorithm design that exploits domain-specific structure. The hybrid approach's success underscores a fundamental principle in combinatorial optimization: theoretical complexity classifications provide essential guidance but should not preclude the development of effective practical solutions for important real-world problems.

The project illustrates the enduring relevance of classical algorithmic techniques—from exact enumeration and greedy heuristics to genetic algorithms and kernelization—while showing how their thoughtful integration can yield solutions superior to any individual approach. As digital labor markets continue to grow and evolve, such algorithmic solutions

will play an increasingly critical role in matching talent with opportunity efficiently and effectively.

Ultimately, this investigation contributes to both the theoretical understanding of covering problems with skill constraints and the practical toolkit available to platform developers, bridging the gap between academic computer science and industrial software engineering in addressing one of the defining optimization challenges of the digital economy.

References

[1] Karp, R. M. (1972). *Reducibility among combinatorial problems*. In Complexity of computer computations (pp. 85-103).

[2] Vazirani, V. V. (2001). *Approximation algorithms*. Springer Science & Business Media.

[3] Talbi, E. G. (2009). *Metaheuristics: from design to implementation*. John Wiley & Sons.

[4] Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. Pearson Education.

[5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[6] Talbi, E. G. (2009). *Metaheuristics: from design to implementation*. John Wiley & Sons.